

Saveetha Institue Of Medical And Technical Sciences

CSA3712-Software Testing

CO2_AT1_Scenario-Based Assessment

Written by:

G Bhumika

192312237

Electronics &Communication Engineering

Questions

Question 1

An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.

Question 2

A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.

Question 3

A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.

Question 4

A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.

Question 5

A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP,

BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.

Question 6

A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.

Question 7

An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.

Question 8

A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.

Question 9

A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.

Question 10

A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.

Question 11

A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.

Question 12

An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.

Question 13

A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.

Question 14

A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.

Question 15

A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.

Answers for the Questions

Answer 1

1. Understanding the scenario

The bank is seeing failures when the transfer amount is exactly on a limit. The question does not give the limits, so I have assumed the following rules to make the test cases concrete.

Rule	Assumed value
Minimum transfer amount	Rs. 100
Maximum transfer amount (per transaction)	Rs. 1,00,000
Decimal places allowed	Up to 2 (paise)
Other condition	Amount must not be more than the available balance

Failures that appear only on the limits usually point to an off-by-one mistake in the code, for example:

```
if (amount > MIN && amount < MAX)    // wrong: rejects 100 and 100000
if (amount >= MIN && amount <= MAX)  // correct
```

2. Evaluation of the current test design

- The team mostly used random amounts and a few "normal" values such as 500 or 5,000. Values like exactly 100 or exactly 1,00,000 were almost never picked.
- There were no invalid classes (zero, negative, text, too many decimals), so error handling was not checked.
- There was no link between the test cases and the requirement, so nobody could say which rule was not tested.
- There was no clear stop condition. Testing ended when time was over and not when the rules were covered.

3. Equivalence Class Partitioning (ECP)

The amount field is divided into groups where the system should behave in the same way. One test per group is enough.

Class	Description	Sample input	Expected result
V1	Valid: 100 to 1,00,000, up to 2 decimals, balance is enough	5000	Transfer successful
I1	Invalid: positive amount below minimum	50	Error: below minimum limit

Class	Description	Sample input	Expected result
I2	Invalid: zero or negative	0, -200	Error: invalid amount
I3	Invalid: above maximum	250000	Error: limit exceeded
I4	Invalid: not a number or empty	abc, (blank)	Error: enter a valid amount
I5	Invalid: more than 2 decimal places	500.555	Error: invalid format
I6	Valid amount but more than the balance	80000 (balance 20000)	Error: insufficient funds

4. Boundary Value Analysis (BVA)

Defects are most common on the edges of a class, so each limit is tested with the value just below, on, and just above it.

Boundary	Test value	Expected result
Min - 1	99 (and 99.99)	Rejected
Min	100	Accepted
Min + 1	101	Accepted
Max - 1	99999	Accepted
Max	100000	Accepted
Max + 1	100001 (and 100000.01)	Rejected

5. Is random testing useful here?

Suppose the team picks amounts randomly between 1 and 2,00,000. The chance of hitting exactly 100 or exactly 1,00,000 in one try is 2 / 2,00,000 = 0.001%. Even after 1,000 random tests the chance of touching a boundary is only about 1%. So random testing is a poor way to find boundary bugs.

Advantages of random testing

- Very cheap and easy to automate.
- Can find unexpected crashes (for example, strange characters in the field).
- Needs no deep knowledge of the requirement.

Disadvantages of random testing

- Does not target boundaries or special rules.
- No measure of how much of the requirement is covered.
- Failures are hard to reproduce unless the random seed is saved.
- Many tests end up checking the same thing again and again.

Decision: random testing should not be the main technique. I would keep it only as a small extra layer (fuzzing and regression runs with a fixed seed) and replace it with ECP + BVA as the base of the test design.

6. Proposed structured and optimized approach

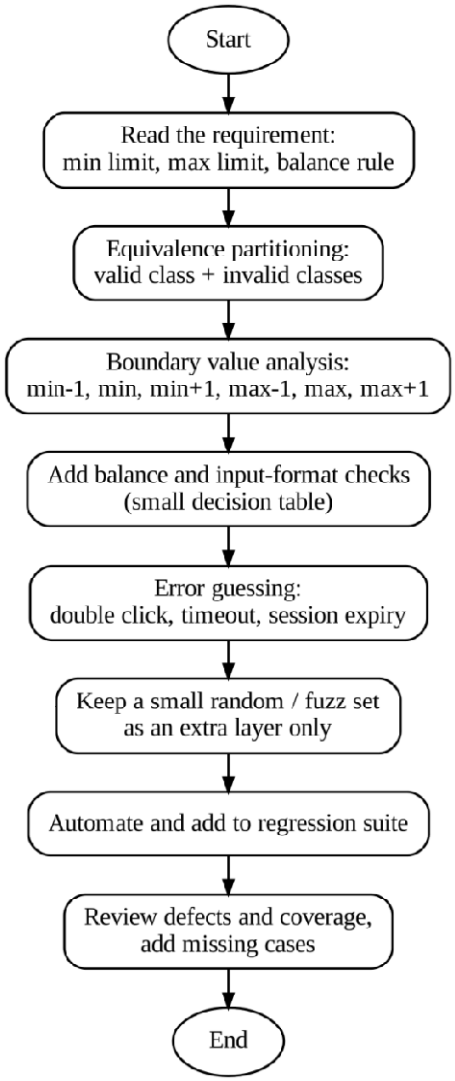


Figure 1.1: Proposed test design flow for the fund transfer module

7. Optimized test set

The set below covers all 7 classes and all 6 boundaries with only 15 test cases.

ID	Technique	Amount	Balance	Expected result
TC1	BVA min - 1	99	50000	Rejected (below minimum)
TC2	BVA min	100	50000	Accepted
TC3	BVA min + 1	101	50000	Accepted
TC4	ECP V1 (nominal)	50000	90000	Accepted

ID	Technique	Amount	Balance	Expected result
TC5	BVA max - 1	99999	200000	Accepted
TC6	BVA max	100000	200000	Accepted
TC7	BVA max + 1	100001	200000	Rejected (limit exceeded)
TC8	ECP I2	0	50000	Rejected (invalid amount)
TC9	ECP I2	-500	50000	Rejected (invalid amount)
TC10	ECP I4	abc	50000	Rejected (invalid input)
TC11	ECP I4	(blank)	50000	Rejected (invalid input)
TC12	ECP I5	500.555	50000	Rejected (invalid format)
TC13	ECP I6	80000	20000	Rejected (insufficient funds)
TC14	Boundary + balance	100000	99999	Rejected (insufficient funds)
TC15	Boundary + balance	5000	5000	Accepted (balance becomes 0)

8. Justification

- **Maximum defect detection:** every limit is tested exactly where off-by-one errors happen, and every invalid class is tried at least once.
- **Optimization:** 15 cases replace hundreds of random ones. Each case has a purpose, so there are no duplicates.
- **Traceability:** every test case is linked to a rule, so any gap is visible.
- **Repeatable:** the cases are fixed, so they can be automated and run after every code change.

Answer 2

1. Understanding the scenario

The module was tested, but serious defects still reached the patients. This normally means that the tests only followed the “happy path” and never forced the special paths such as invalid data, allergy blocking or the safety limit on the dose. For the analysis I have taken a simple conceptual module that calculates the medicine dose for a patient.

2. Conceptual code and Control Flow Graph (CFG)

```
[1]  read(age, weight, allergy)
[2]  if (age <= 0 || weight <= 0)
[3]      return "INVALID"
[4]  if (allergy == true)
[5]      return "BLOCKED"
[6]  if (age < 12)
[7]      dose = weight * 5
     else
[8]      dose = weight * 10
[9]  if (dose > MAX)      // MAX = 500
[10]     dose = MAX
[11] return dose
```

Shaded nodes (2, 4, 6, 9) are decisions.
Nodes 3, 5 and 11 go to the Exit node.

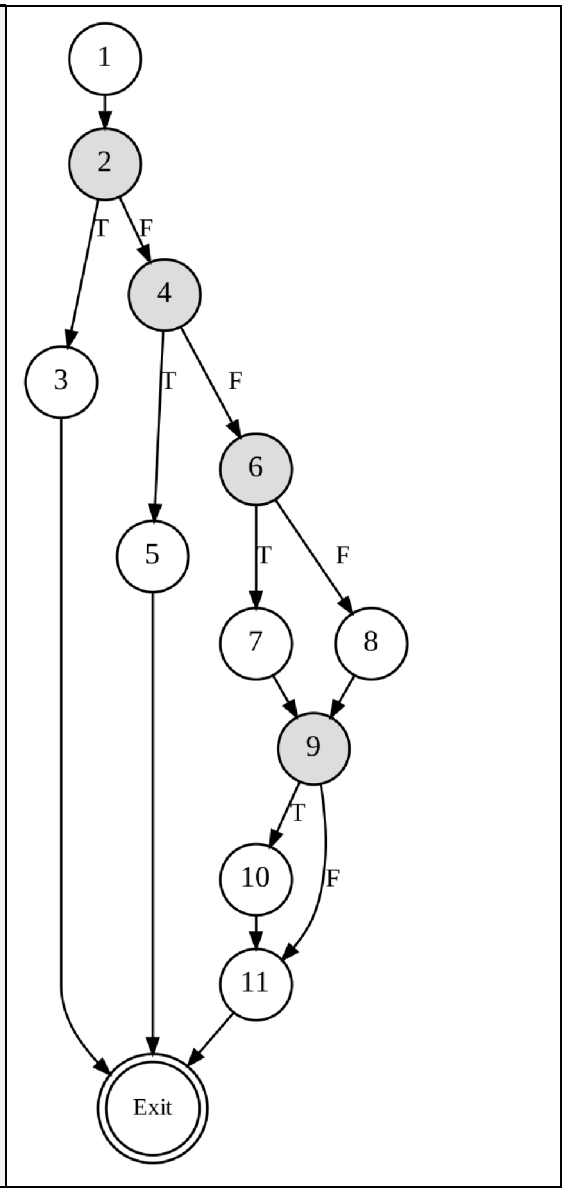


Figure 2.1: Code with node numbers and its control flow graph

Cyclomatic complexity: edges E = 15, nodes N = 12 (including Exit), so V(G) = E - N + 2 = 15 - 12 + 2 = **5**. Check by decision count: 4 decisions + 1 = 5. So at least 5 independent paths exist. Counting every possible combination there are 6 complete paths.

3. Evaluation of the testing done

Assume the team ran only two typical tests: (age 35, weight 50, no allergy) and (age 8, weight 20, no allergy). Both are normal patients with no problem.

Coverage type	Formula	Result achieved
Statement coverage	Statements executed / total statements x 100	8 / 11 = 72.7% (nodes 3, 5, 10 missed)

Coverage type	Formula	Result achieved
Branch coverage	Branches executed / total branches x 100	5 / 8 = 62.5% (2-True, 4-True, 9-True missed)
Path coverage	Paths executed / total paths x 100	2 / 6 = 33.3%

The three statements that were never run are exactly the three safety checks: invalid data (node 3), allergy block (node 5) and dose cap (node 10). That is why the critical defects were found only after deployment. Also, the team had no test adequacy criterion such as "100% branch coverage", so nobody noticed the gap.

4. Missing paths

Path	Route in the CFG	Meaning	Status
A	1-2-3-Exit	Invalid age or weight	Missed
B	1-2-4-5-Exit	Allergy, dose blocked	Missed
C	1-2-4-6-7-9-11-Exit	Child, dose below MAX	Tested
D	1-2-4-6-7-9-10-11-Exit	Child, dose capped	Missed
E	1-2-4-6-8-9-11-Exit	Adult, dose below MAX	Tested
F	1-2-4-6-8-9-10-11-Exit	Adult, dose capped	Missed

5. Improved testing strategy

The aim is 100% statement, 100% branch and 100% path coverage (only 6 paths, so this is practical), plus boundary values on the conditions. The compound condition in node 2 is tested so that each part is shown to affect the result alone.

ID	Age	Weight	Allergy	Expected output	Covers
T1	0	60	No	INVALID	Path A, age part of node 2
T2	30	0	No	INVALID	Path A, weight part of node 2
T3	30	60	Yes	BLOCKED	Path B
T4	8	20	No	100	Path C
T5	8	120	No	500 (capped)	Path D
T6	30	40	No	400	Path E
T7	30	80	No	500 (capped)	Path F
T8	11	50	No	250	Boundary: age just below 12
T9	12	50	No	500	Boundary: age 12 and dose = MAX

ID	Age	Weight	Allergy	Expected output	Covers
T10	12	50.1	No	500 (501 capped)	Boundary: dose just above MAX

T1, T2 and T4 together also give MC/DC for the condition in node 2: (age<=0 true, weight<=0 false), (false, true) and (false, false).

6. How this improves reliability and reduces risk

- Every safety rule (allergy block, dose cap, input check) has at least one test, so the most dangerous failures are checked before release.
- Coverage is now measured against a clear target, so missing paths are found during testing and not by patients.
- Boundary tests catch off-by-one mistakes such as age <= 12 written instead of age < 12.
- The CFG and the path table can be reused whenever the code changes, which keeps regression testing quick and complete.

Answer 3

1. Understanding the scenario

The e-commerce system calculates the order total from price, quantity and discount. Testers checked each field on its own, but errors appeared when the fields were used together. The exact rules are not given, so I assumed these:

Field	Valid range	Extra rule
Price	Rs. 1 to Rs. 10,000	-
Quantity	1 to 100	-
Discount	0% to 50%	More than 10% is allowed only when quantity is 10 or more (bulk order)

Total = price x quantity x (1 - discount / 100).

2. Limitations of the current approach

- Fields were tested one at a time, so the rule that links discount and quantity was never exercised.
- Only typical values were used, so boundaries such as quantity 9 and 10 were never tried.
- Trying every combination is not possible. With only 5 values per field there are 5 x 5 x 5 = 125 cases, and real values give thousands.
- There was no table or model of the business rules, so missing combinations went unnoticed.

3. Equivalence Class Partitioning

Field	Valid classes	Invalid classes
Price	1 to 10000	Less than 1 (0, negative); more than 10000; non-numeric
Quantity	1 to 9 (small order); 10 to 100 (bulk order)	Less than 1; more than 100; decimal such as 2.5; non-numeric
Discount	0 to 10 (allowed for all orders); 11 to 50 (bulk only)	Less than 0; more than 50; non-numeric

4. Boundary Value Analysis

Field	Invalid below	Min	Min + 1	Max - 1	Max	Invalid above
Price	0	1	2	9999	10000	10001
Quantity	0	1	2	99	100	101
Discount	-1	0	1	49	50	51

There is one more boundary because of the bulk rule: quantity 9 and quantity 10 with a discount above 10%. A normal BVA for 3 variables needs 4n + 1 = 13 cases. Robust BVA needs 6n + 1 = 19. Both are far fewer than the 125 combinations.

5. Decision table testing

The discount rule depends on more than one input, so a decision table is used. Y = true, N = false, - = does not matter.

Condition / action	R1	R2	R3	R4	R5	R6
C1: Price valid (1 to 10000)	N	Y	Y	Y	Y	Y
C2: Quantity valid (1 to 100)	-	N	Y	Y	Y	Y
C3: Discount in 0 to 50	-	-	N	Y	Y	Y
C4: Discount more than 10	-	-	-	N	Y	Y
C5: Quantity 10 or more	-	-	-	-	Y	N
Action	Reject: price error	Reject: qty error	Reject: discount error	Accept	Accept	Reject: bulk only

6. Revised test design

Nominal values used in the cases: price 500, quantity 20, discount 5% (all valid).

ID	Source	Price	Qty	Disc %	Expected result
1	BVA price min	1	20	5	Accept, total 19.00
2	BVA price max	10000	20	5	Accept, total 190000.00
3	Invalid price	0	20	5	Reject (R1)
4	Invalid price	10001	20	5	Reject (R1)
5	BVA qty min	500	1	5	Accept, total 475.00
6	BVA qty max	500	100	5	Accept, total 47500.00
7	Invalid qty	500	0	5	Reject (R2)
8	Invalid qty	500	101	5	Reject (R2)
9	BVA discount min	500	20	0	Accept, total 10000.00
10	BVA discount max	500	20	50	Accept, total 5000.00
11	Several errors	0	0	60	Reject, errors shown clearly

7. Justification

- **Defect detection:** single-field boundaries find range errors, and cases 13 to 17 find the combination error between quantity and discount that the old tests missed.
- **Completeness:** all 6 rules of the decision table, all invalid classes and all boundaries are covered.
- **Efficiency:** 18 cases instead of 125 or more, which is about 86% fewer, without losing any rule.
- **Structure:** the decision table is a clear model. When a new rule is added, only a new column is needed.
- If the number of fields grows, pairwise (all-pairs) testing can be added to keep the combinations small.

Answer 4

1. Understanding the scenario

The module has nested conditions inside a loop, and the team reports only partial path coverage. A loop can run any number of times, so the number of paths is very large. Testing every path is not possible. Instead we find the independent paths (basis paths) and test each of them, and then test the loop boundaries separately. I use a simple conceptual module that processes a list of numbers.

2. Code and Control Flow Graph

```
[1] read n; i = 1
[2] while (i <= n) {
[3]     read x
        if (x > 0) {
[4]         if (x % 2 == 0)
[5]             evenSum += x
            else
[6]             oddSum += x
        } else
[7]         negCount++
[8]     i++
    }
[9] print results

Decision nodes: 2, 3, 4 (shaded).
```

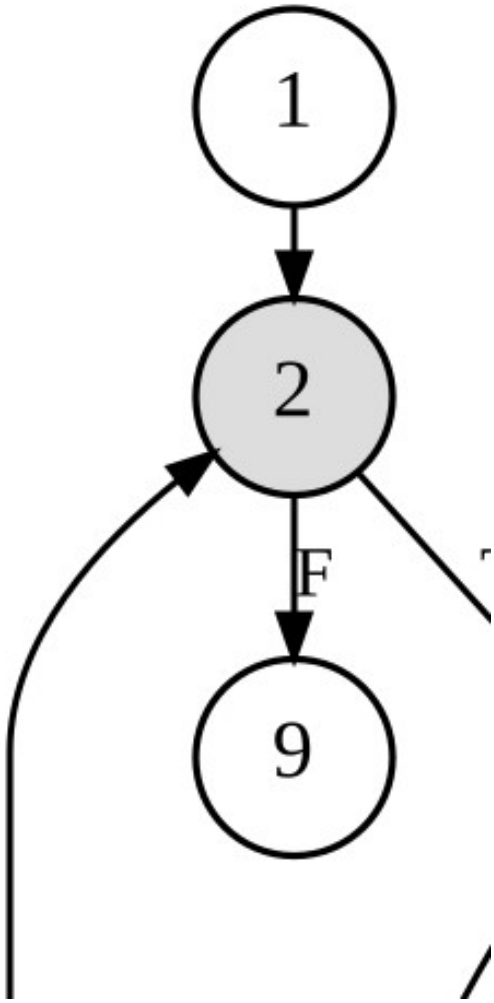


Figure 4.1: CFG of the nested loop module

Cyclomatic complexity: E = 11, N = 9, so V(G) = 11 - 9 + 2 = **4**. Decision nodes = 3, and 3 + 1 = 4. So there are 4 independent paths.

3. Evaluation of the current approach and the gaps

Assume the team ran one test: n = 3 with inputs 4, 7 and 10 (all positive). It looks like a good test because the loop runs three times, but it only touches the positive branches.

Measure	**Result with the old test**	**What was missed**
Statement coverage	8 / 9 = 88.9%	Node 7 (negative or zero input)
Branch coverage	5 / 6 = 83.3%	Node 3 false branch
Independent paths	2 / 4 = 50%	Path P1 (loop skipped) and P2 (non-positive input)
Loop boundaries	Only n = 3 tried	n = 0, n = 1, n = 2, maximum value

Impact of missing independent paths: every path that is not run is a place where a defect can hide. Here, the zero-iteration case could crash on an empty file, and the negative branch could count zeros wrongly. Since the statement coverage looked high (88.9%), the team believed the testing was nearly complete, which is the real danger of partial path coverage.

4. Independent (basis) paths

Path	Route	Test data	Expected result
P1	1-2-9	n = 0	All totals are 0, loop skipped
P2	1-2-3-7-8-2-9	n = 1, x = -5	negCount = 1
P3	1-2-3-4-5-8-2-9	n = 1, x = 4	evenSum = 4
P4	1-2-3-4-6-8-2-9	n = 1, x = 7	oddSum = 7

5. Complete path testing strategy

Beyond the 4 basis paths, the loop is tested at its boundaries, and different branches are mixed inside the loop.

ID	Purpose	Input (n; values)	Expected result
T1	P1, loop skipped	0; none	evenSum 0, oddSum 0, negCount 0
T2	P2, one pass	1; -5	negCount 1
T3	P3, one pass	1; 4	evenSum 4
T4	P4, one pass	1; 7	oddSum 7
T5	Zero value (x > 0 boundary)	1; 0	negCount 1 (zero is not positive)
T6	Two passes, even then negative	2; 4, -3	evenSum 4, negCount 1
T7	Two passes, odd then even	2; 7, 8	oddSum 7, evenSum 8
T8	Typical mix	5; 3, -2, 6, 0, 9	oddSum 12, evenSum 6, negCount 2
T9	Maximum - 1 and maximum	MAX_N - 1 and MAX_N; all positive even	Correct totals, no crash
T10	Above maximum	MAX_N + 1	Rejected with an error

6. Justification

- **Coverage criteria:** the strategy reaches 100% statement, 100% branch and 100% independent-path coverage, which is stronger than the 50% path coverage we started with.
- **Cyclomatic complexity** gives the minimum number of paths to test (4), so the effort is based on a number and not on a guess.
- **Loop testing** (skip, once, twice, typical, maximum, above maximum) covers the cases that path testing alone cannot, because a loop has unlimited paths.
- **Expected outcome:** every critical path has a known input and a known result, so a defect shows up as a mismatch that can be traced to a single path.

Answer 5

1. Understanding the scenario

The financial application already uses both black-box and white-box testing, but defects still reach production. So the problem is not the lack of techniques. The problem is the way they were combined and measured.

2. Critical analysis of the current integration

Observation	Problem it causes
Black-box and white-box tests were done separately, often by different people	The same cases get repeated, and other cases are left out. Nobody checks how one supports the other.
Black-box cases came only from the requirement document	Paths in the code that the requirement does not mention (error handling, rare conditions) stay untested.
White-box testing stopped at statement coverage	Statement coverage can be 100% while many branches and conditions are never tried.
Coverage was not fed back into test design	Gaps were not used to add new test cases.
No exit criteria	Testing ended on a date and not when the quality target was met.

3. Were adequacy criteria and coverage metrics applied properly?

No. The metrics below are the usual ones. In this project they were either not measured, or only the weakest one was used.

Metric	Formula	Proper use
Statement coverage	Executed statements / total statements x 100	Minimum level only

Metric	Formula	Proper use
Branch coverage	Executed branches / total branches x 100	Target 100% for money-related modules
Path coverage	Executed independent paths / V(G)	All basis paths must run
Cyclomatic complexity	V(G) = E - N + 2 (or decisions + 1)	Gives the minimum number of tests
Black-box adequacy	Classes and boundaries tested / total	All classes and all boundaries

4. Proposed hybrid strategy

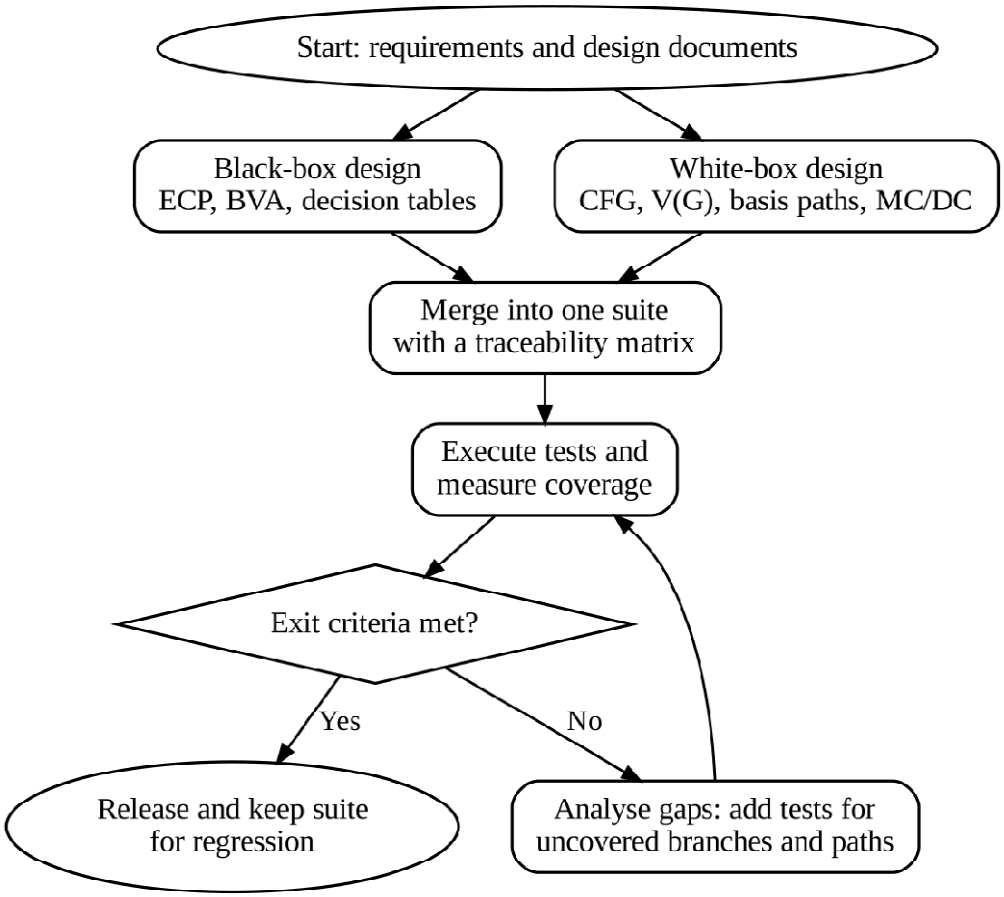


Figure 5.1: Hybrid test design process with a coverage feedback loop

1. **Black-box design first:** apply ECP and BVA to every input and a decision table for rules that depend on more than one input.
2. **White-box design in parallel:** draw the CFG, find V(G) and the basis paths, and list the compound conditions for MC/DC.
3. **Merge and trace:** put all cases in one suite. Each case is linked to a requirement and to the path or condition it covers.

4. **Execute and measure:** use a coverage tool to measure statement, branch and path coverage.
5. **Close the gaps:** every uncovered branch or path gets a new test case, or is shown to be impossible.
6. **Exit criteria:** all classes and boundaries tested, 100% branch coverage, all basis paths executed, MC/DC on critical conditions, and no open high-severity defect.

5. Worked example: loan interest module

```
[1] read(amount, score)
[2] if (amount < 10000 || amount > 500000)
[3]     return REJECT
[4] if (score >= 750)      rate = 8
[5] else if (score >= 650) rate = 10
[6] else                   rate = 14
[7] interest = amount * rate / 100
[8] return interest
```

Decisions: 3 (lines 2, 4, 5), so V(G) = 3 + 1 = 4 basis paths: reject, score 750 or more, score 650 to 749, and score below 650.

ID	Amount	Score	Expected	Black-box reason	White-box reason
M1	9999	700	REJECT	BVA min - 1	Path 1, condition A true
M2	10000	700	1000 (rate 10)	BVA min	Path 3, both conditions false
M3	500000	800	40000 (rate 8)	BVA max, ECP score 750+	Path 2
M4	500001	700	REJECT	BVA max + 1	Condition B true
M5	50000	649	7000 (rate 14)	BVA score - 1	Path 4
M6	50000	650	5000 (rate 10)	BVA score boundary	Branch at line 5 true
M7	50000	749	5000 (rate 10)	BVA score upper edge	Branch at line 4 false
M8	50000	750	4000 (rate 8)	BVA score boundary	Branch at line 4 true

Eight cases give all boundary values, all classes, all 4 basis paths, 100% branch coverage and MC/DC for line 2.

6. How this ensures better coverage and defect prevention

- Black-box cases check that the software does what the requirement says. White-box cases check what the code actually does, including parts the requirement never mentions. Together they cover both.
- The feedback loop means coverage numbers decide when to stop, so gaps are found before release.
- Traceability shows exactly which rule or path is behind each test, which helps in audits of financial software.
- Defects are prevented as well as found, because reviewing the CFG and the decision table exposes logic errors early.

7. Conclusion

Using both kinds of testing was a good start, but without shared design, proper coverage metrics and exit criteria they did not give protection. A hybrid strategy where ECP and BVA decide the inputs, CFG and path testing decide the structure, and coverage results drive new test cases gives measurable completeness and fewer production defects.

Answer 6

1. Understanding the scenario

The bonus depends on two inputs, years of service and performance rating. Employees with exactly 5, 10 and 15 years got wrong payouts. These are the exact points where the bonus slab changes, so the defect is a boundary problem. The question does not give the rules, so I assumed the following (salary Rs. 50,000 in the examples).

Years of service	Bonus slab (% of salary)
0 to 4	2%
5 to 9	5%
10 to 14	8%
15 and above (maximum 40)	12%

Performance rating	Multiplier on the slab bonus
1 or 2	0 (no bonus)
3	1.0
4	1.25
5	1.5

2. Analysis of the failure

A common coding mistake is to write the slab conditions with > instead of >=:

```
if (years < 5)                    pct = 2;
else if (years > 5 && years < 10)   pct = 5;   // should be years >= 5
else if (years > 10 && years < 15)  pct = 8;   // should be years >= 10
else if (years > 15)              pct = 12;  // should be years >= 15
else                              pct = 0;   // 5, 10 and 15 end up here
```

With this code, an employee with exactly 5, 10 or 15 years falls into the last else and gets no bonus. Any test with a value inside a slab (3, 7, 12, 20) passes, so the mistake stays hidden.

3. Evaluation of the current test design

- The old tests used values from the middle of each slab (3, 7, 12 and 20 years), shown as hollow circles in Figure 6.1. None of them sits on a boundary.
- Only one input was varied at a time, and the rating was always a typical value.
- There were no invalid inputs such as negative years or a rating of 0 or 6.
- Test values were chosen by feel, so there was no rule that said when the testing was complete.

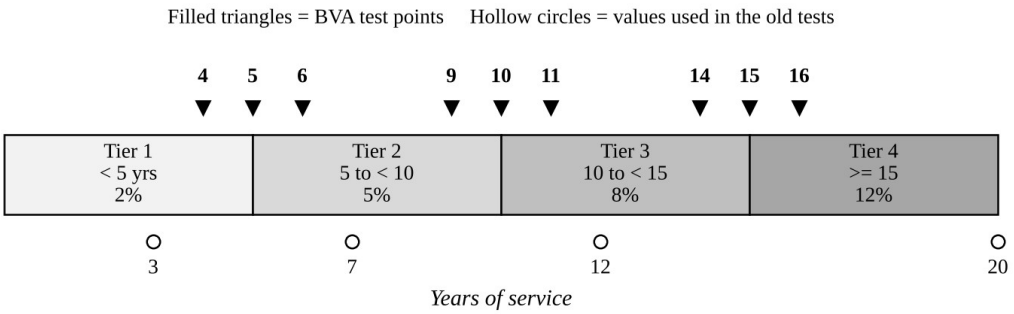


Figure 6.1: Bonus slabs with the old test values and the new BVA test points

4. Equivalence Class Partitioning

Input	Class	Range	Representative value
Years	Invalid (low)	less than 0	-1
Years	Slab 1	0 to 4	2
Years	Slab 2	5 to 9	7
Years	Slab 3	10 to 14	12
Years	Slab 4	15 to 40	25
Years	Invalid (high)	more than 40	41
Rating	Invalid (low)	less than 1	0
Rating	No bonus	1 to 2	2
Rating	Normal / Good / Excellent	3 / 4 / 5	3, 4, 5

Input	Class	Range	Representative value
Rating	Invalid (high)	more than 5	6

5. Boundary Value Analysis

Each slab edge is tested with the value below, on and above it. Rating is fixed at 3 (multiplier 1.0), so the expected bonus is just the slab percentage of 50,000.

Years	Slab	Expected bonus (Rs.)
-1	Invalid	Error
0, 1, 4	Slab 1	1000
5, 6, 9	Slab 2	2500
10, 11, 14	Slab 3	4000
15, 16, 40	Slab 4	6000
41	Invalid	Error

6. Redesigned test suite

ID	Years	Rating	Expected bonus (Rs.)	Purpose
TS1	0	3	1000	Lowest valid years
TS2	4	3	1000	Just below 5
TS3	5	3	2500	Boundary 5 (failed earlier)
TS4	6	3	2500	Just above 5
TS5	9	3	2500	Just below 10
TS6	10	3	4000	Boundary 10 (failed earlier)
TS7	11	3	4000	Just above 10
TS8	14	3	4000	Just below 15
TS9	15	3	6000	Boundary 15 (failed earlier)
TS10	16	3	6000	Just above 15
TS11	40	3	6000	Maximum valid years
TS12	-1	3	Error	Invalid years
TS13	41	3	Error	Invalid years
TS14	7	1	0	Rating class: no bonus
TS15	7	2	0	Rating boundary below 3

ID	Years	Rating	Expected bonus (Rs.)	Purpose
TS16	7	3	2500	Rating 3
TS17	7	4	3125	2500 x 1.25
TS18	7	5	3750	2500 x 1.5
TS19	7	0	Error	Invalid rating
TS20	7	6	Error	Invalid rating
TS21	5	5	3750	Years boundary with top rating
TS22	10	4	5000	4000 x 1.25
TS23	15	5	9000	6000 x 1.5
TS24	15	2	0	Years boundary with no-bonus rating

7. Why BVA and ECP are better than exploratory testing here

Point	BVA + ECP	Exploratory testing
Type of logic	Fixed thresholds (5, 10, 15) are exactly what BVA is made for	Depends on the tester thinking of the thresholds
Repeatability	Same cases every time, easy to automate	Different every session, hard to repeat
Coverage	Every class and every boundary can be counted	No clear measure of what was covered
Money risk	Wrong payouts are found before payroll runs	A boundary may be missed, and employees get the wrong money
Audit	Test cases are documents that auditors can check	Notes are informal and hard to audit
Regression	Re-run after every rule change at low cost	Needs a person each time

Exploratory testing is still useful for usability and for things nobody thought of, but for exact financial rules the systematic techniques are more reliable. A single missed boundary here means real money paid incorrectly.

Answer 7

1. Understanding the scenario

The laboratory controller decides whether to run the heater or cooler from the temperature, and it raises an alarm when something is unsafe. After release the alarm did not sound in one sensor-malfunction sequence. This suggests that the branch that handles a faulty sensor was never executed in testing. I use the following conceptual logic (LOW = 20, HIGH = 80).

2. Code and Control Flow Graph

```
[1]  read(t, s1ok, s2ok)
[2]  if (!s1ok && !s2ok)
[3]      alarm = ON; stop()
     else {
[4]    if (!s1ok || !s2ok)
[5]        useBackup = true
[6]    if (t < LOW)
[7]        heater = ON
[8]    else if (t > HIGH)
[9]        cooler = ON
[10]   else heater = cooler = OFF
[11]   if (useBackup && t > HIGH)
[12]       alarm = ON
     }
[13] return

Decisions: D1 = node 2, D2 = node 4,
D3 = node 6, D4 = node 8, D5 = node 11
```

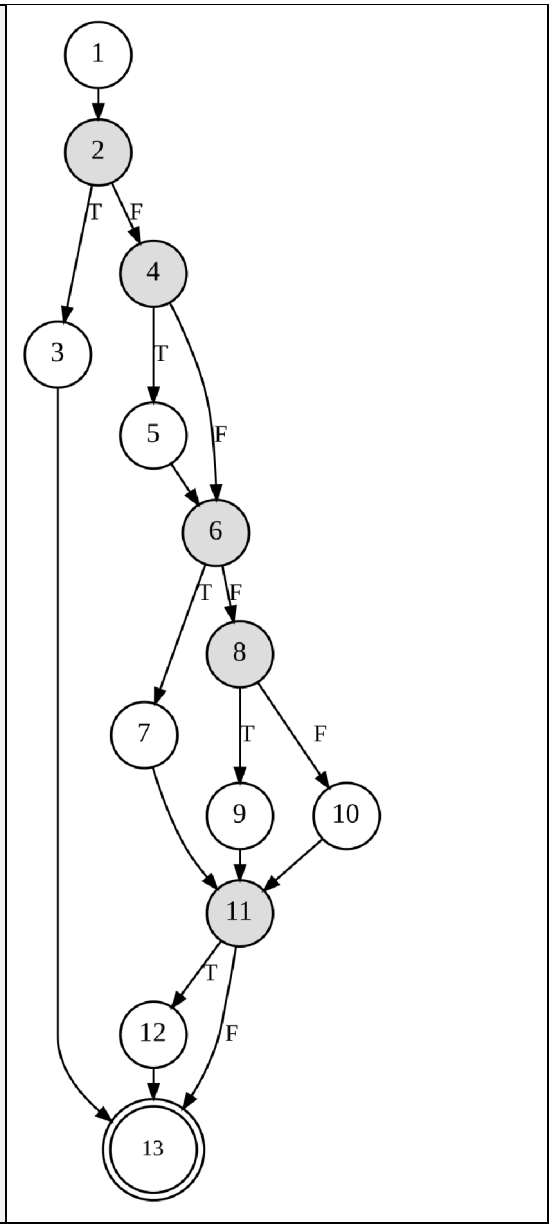


Figure 7.1: Code with node numbers and its CFG

Nodes N = 13, edges E = 17, so V(G) = 17 - 13 + 2 = **6**. Check: 5 decisions + 1 = 6.

3. Gaps in branch coverage

Assume the old tests were: (both sensors faulty), (both OK, t = 10), (both OK, t = 50) and (both OK, t = 90). All of them are cases where the sensors work.

Decision	True branch	False branch	Comment
D1: !s1ok && !s2ok	Covered	Covered	-
D2: !s1ok || !s2ok	NOT covered	Covered	No test with exactly one faulty sensor
D3: t < LOW	Covered	Covered	-
D4: t > HIGH	Covered	Covered	-
D5: useBackup && t > HIGH	NOT covered	Covered	The alarm for a backup sensor is never triggered

Statement coverage = 10 / 12 = 83.3% (nodes 5 and 12 never run). Branch coverage = 7 / 10 = 70%. The two missed branches are exactly the sensor-malfunction alarm path, which explains the failure after release. Even if branch coverage had been 100%, the compound conditions in D1, D2 and D5 would still not be fully checked, which is why MC/DC is needed.

4. White-box strategy for 100% MC/DC

MC/DC (Modified Condition/Decision Coverage) means: every decision takes both outcomes, every condition takes both values, and every condition is shown to change the decision result on its own while the other conditions stay fixed. For n conditions, n + 1 tests are enough, whereas full combinations need 2^n.

ID	s1ok	s2ok	t	Expected behaviour
T1	F	F	50	Alarm ON and system stops
T2	F	T	50	Backup sensor used, outputs OFF, no alarm
T3	T	F	50	Backup sensor used, outputs OFF, no alarm
T4	T	T	10	Heater ON
T5	T	T	50	Heater and cooler OFF
T6	T	T	90	Cooler ON, no alarm
T7	F	T	90	Backup used, cooler ON, alarm ON

Pairs that show each condition independently changes the outcome (A and B are the two conditions of a decision):

Decision	Condition	Test pair	What changes
D1: A && B (A = !s1ok, B = !s2ok)	A	T1 and T3	A true -> false, B stays true, result true -> false
	B	T1 and T2	B true -> false, A stays true, result true -> false
D2: A || B	A	T2 and T5	A true -> false, B stays false, result true -> false
	B	T3 and T5	B true -> false, A stays false, result true -> false
D3: t < LOW	t < LOW	T4 and T5	Single condition: true and false
D4: t > HIGH	t > HIGH	T6 and T5	Single condition: true and false
D5: useBackup && t > HIGH	useBackup	T7 and T6	useBackup true -> false, t > HIGH stays true
	t > HIGH	T7 and T2	t > HIGH true -> false, useBackup stays true

These 7 tests give 100% statement, 100% branch and 100% MC/DC. T7 is the test that would have caught the missing alarm.

5. Impact on safety-critical reliability

- **Hidden conditions are exposed.** In a && b the second part may never matter in ordinary tests. MC/DC proves that each sensor condition really controls the alarm.
- **Failure sequences are covered.** One sensor failing, the other failing, and both failing are each tested together with high temperature, which are the cases that matter in a laboratory.
- **Cost stays small.** n + 1 tests per decision (7 tests here) instead of every combination.
- **Industry practice.** MC/DC is required for the highest safety level of DO-178C (avionics) and is highly recommended at the top level of ISO 26262 (automotive). Laboratory control with a safety alarm deserves the same discipline.
- **Evidence.** The test table and pairs form a record that the safety logic was checked, which supports certification and audits.

Answer 8

1. Understanding the scenario

The portal lets a user apply more than one promo code, and some combinations make the Total Amount negative. The exact rules are not given, so I assumed these (all values in Rs.):

Item	Assumption
Percentage promo (PCT)	30% off the base price, only one allowed per booking
Flat promo (FLAT)	Rs. 1,000 off, only one allowed per booking
User tier	Regular: no extra discount. Premium: extra 10% (added to any percentage discount)
Order of calculation	Total = price x (1 - percentage discounts) - flat promo
Required rule	Total must never be below 0. If discounts are bigger than the price, the total is capped at 0

2. Limitations of testing the inputs in isolation

- A single promo on its own works, the Premium tier on its own works, and the price on its own works. The bug appears only when a flat promo, a low price and other discounts are present together.
- Example: price 800 with the flat promo alone gives 800 - 1000 = -200. No single-field test would ever try this.
- The number of combinations grows quickly. With 5 price values, 3 options for each promo slot and 3 tiers, there are 5 x 3 x 3 x 3 = 135 cases, and a plain list of them contains many repeats.

3. Cause-effect graph

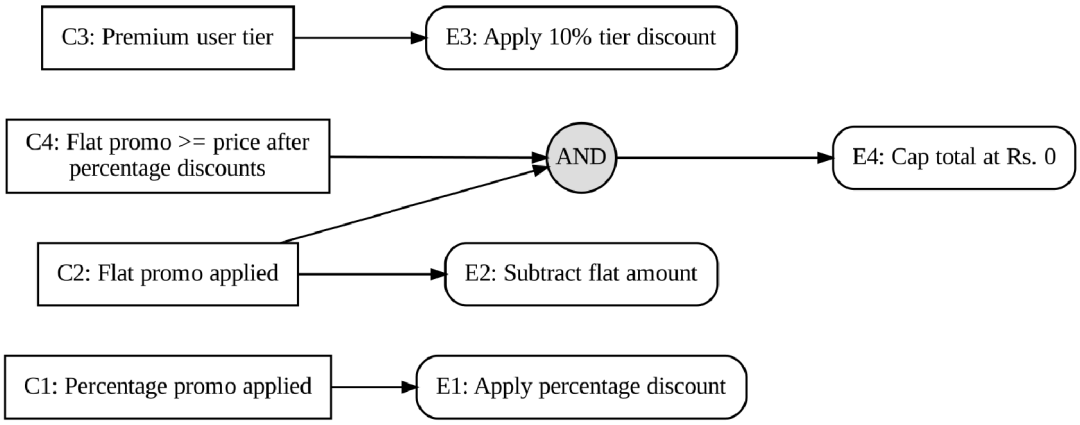


Figure 8.1: Cause-effect graph for the promo rules

C4 is a derived condition: the flat promo is bigger than or equal to the price left after percentage discounts. When C2 and C4 are true together, the total must be capped (E4). This AND relation is the one that was never tested.

4. Decision table

Rule	C1: PCT promo	C2: FLAT promo	C3: Premium	C4: FLAT >= net price	Total Amount
R1	N	N	N	-	P
R2	N	N	Y	-	P x 0.90
R3	Y	N	N	-	P x 0.70
R4	Y	N	Y	-	P x 0.60
R5	N	Y	N	N	P - 1000
R6	N	Y	N	Y	0 (capped)
R7	N	Y	Y	N	P x 0.90 - 1000
R8	N	Y	Y	Y	0 (capped)
R9	Y	Y	N	N	P x 0.70 - 1000
R10	Y	Y	N	Y	0 (capped)
R11	Y	Y	Y	N	P x 0.60 - 1000
R12	Y	Y	Y	Y	0 (capped)

A full table would have 16 columns (2 x 2 x 2 x 2). Because C4 is meaningless when no flat promo is applied, four of them merge, leaving **12 rules**.

5. Optimized test set

ID	Price	Promos	Tier	Expected total
R1	5000	None	Regular	5000
R2	5000	None	Premium	4500
R3	5000	PCT	Regular	3500
R4	5000	PCT	Premium	3000
R5	5000	FLAT	Regular	4000
R6	800	FLAT	Regular	0 (not -200)
R7	5000	FLAT	Premium	3500
R8	1000	FLAT	Premium	0 (not -100)
R9	5000	PCT + FLAT	Regular	2500
R10	1000	PCT + FLAT	Regular	0 (not -300)

ID	Price	Promos	Tier	Expected total
R11	5000	PCT + FLAT	Premium	2000
R12	1500	PCT + FLAT	Premium	0 (not -100)
B1	1000	FLAT	Regular	0 (discount equals price)
B2	1001	FLAT	Regular	1 (just above the cap)
I1	5000	Same code applied twice	Regular	Second code rejected, total 4000
I2	5000	Two PCT codes	Regular	Second code rejected, total 3500
I3	5000	Expired code	Regular	Code rejected, total 5000
I4	0 or -100	Any	Any	Booking rejected (invalid price)

R = decision table rule, B = boundary of the cap, I = invalid combination.

6. Justification

- **Defect detection:** R6, R8, R10, R12, B1 and B2 target the negative-total defect directly, and the other rules check that the normal calculations are right.
- **Valid and invalid combinations:** all 12 valid rules and the 4 invalid cases (duplicate, stacking, expired, bad price) are mapped.
- **Minimum redundancy:** 18 cases instead of 135, which is about 87% less, and every case tests a different rule.
- **Traceability:** each test points to a rule, so when a promo rule changes only the matching rows need to be updated.

Answer 9

1. Understanding the scenario

A legacy migration tool loops through tables and, inside that, through the rows of each table. The team reports high statement coverage, but the tool hangs in production. High statement coverage only tells us that every line was run once. It says nothing about whether every loop actually ends. I use a conceptual version of the tool that contains one such defect.

2. Code and Control Flow Graph

```
[1]  i = 0
[2]  while (i < tables) {
[3]     j = 0
[4]     while (j < rows[i]) {
[5]        if (valid(row[i][j]))
[6]           migrate(row[i][j])
[7]        j++
       }
[8]     if (rows[i] > 0)   // BUG
[9]        i++
    }
[10] end
```

Decisions: nodes 2, 4, 5, 8.
Red dashed edge 8 -> 2: an empty table returns to the loop test without i++, so the loop never ends.

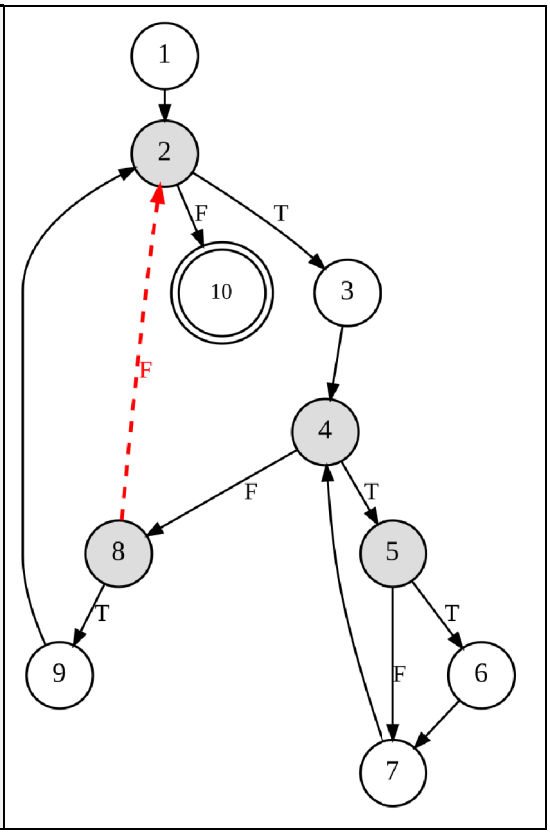


Figure 9.1: CFG of the nested loop module

The defect is that i++ is done only for tables that have rows. For a table with 0 rows, i never changes, so the outer loop repeats forever.

3. Why statement coverage was not enough

Assume the team used one test: 2 tables with 3 and 2 rows, with one invalid row among them.

Measure	Result	Explanation
Statement coverage	10 / 10 = 100%	Every line from 1 to 10 is executed at least once
Branch coverage	7 / 8 = 87.5%	The false branch of node 8 (empty table) is never taken
Independent paths	3 / 5 = 60%	Paths P1 (no tables) and P4 (empty table) are not run
Loop boundaries	Not checked	Zero rows and zero tables were never tried

Statement coverage only asks “was this line executed?”. The faulty path is made of lines that were all executed already, but in a different order. It also does not check termination, because a test that ends is counted as good and a hanging test is only noticed if there is a time limit.

4. Cyclomatic complexity and independent paths

E = 13 edges, N = 10 nodes, so **V(G) = 13 - 10 + 2 = 5**. Check: decisions 2, 4, 5 and 8 give 4 + 1 = 5. At least 5 tests are needed for independent path coverage, but only 1 test was needed for the statement coverage the team reported. The extra 4 are where the problem hid.

Path	Route	Test data	Expected result
P1	1-2-10	tables = 0	Finishes immediately, nothing migrated
P2	1-2-3-4-5-6-7-4-8-9-2-10	tables = 1, rows = 1 (valid)	1 row migrated, finishes
P3	1-2-3-4-5-7-4-8-9-2-10	tables = 1, rows = 1 (invalid)	0 migrated, 1 logged, finishes
P4	1-2-3-4-8-2 ...	tables = 1, rows = 0	Must finish. Hangs in current code (defect found)
P5	1-2-3-4-5-6-7-4-5-6-7-4-8-9-2-10	tables = 1, rows = 2 (valid)	2 rows migrated, inner loop runs twice

5. Loop boundary testing strategy

For nested loops the usual method is to test the innermost loop first while the outer loop is at its minimum, and then move outward.

Loop	Values to test	Reason
Inner loop (rows)	0, 1, 2, typical, MAX - 1, MAX (outer loop = 1 table)	Skip, single pass, repeat, normal load and upper limit
Outer loop (tables)	0, 1, 2, typical, MAX - 1, MAX (each table has 1 row)	Same checks for the outer loop
Mixed	tables = 3 with rows = [2, 0, 2]	Empty table in the middle (hangs in current code)
Beyond limit	MAX + 1	Must be rejected or handled safely

Every test case also runs with a time limit (for example 5 seconds). A test that does not finish in time is marked as failed, so an infinite loop is reported as a defect and does not freeze the test run.

6. Justification using cyclomatic complexity

- V(G) = 5 tells us the minimum number of tests, so the effort is based on a number. It also shows that the earlier suite (statement coverage, 1 test) was far too small.

- Common risk guide for V(G): 1 to 10 simple, 11 to 20 moderate risk, 21 to 50 high risk, above 50 very high risk and almost untestable. Deeply nested legacy code usually scores high, so modules above 10 should be tested path by path and refactored.
- Independent paths plus loop boundaries give 100% branch coverage and finite, checkable tests for every loop.
- Expected outcome: the empty-table path fails on the first run, the defect is fixed (always do i++), and the same test stays in the regression suite.

Answer 10

1. Understanding the scenario

The tracker shows the order state: Ordered, Preparing, Out for Delivery and Delivered. When a driver cancels in the middle of delivery, the app gets stuck. The most likely reason is that the design has no state for this situation, so the event "driver cancels" has nowhere to go.

2. Evaluation of the current black-box strategy

- The tests followed only the happy path Ordered, Preparing, Out for Delivery, Delivered.
- Inputs were tested one by one with ECP and BVA, with no thought about the order of events.
- Events like cancellation by the driver or the customer were not modelled for every state.
- Invalid events (for example, delivering an order that was never picked up) were not tried.

3. State Transition model

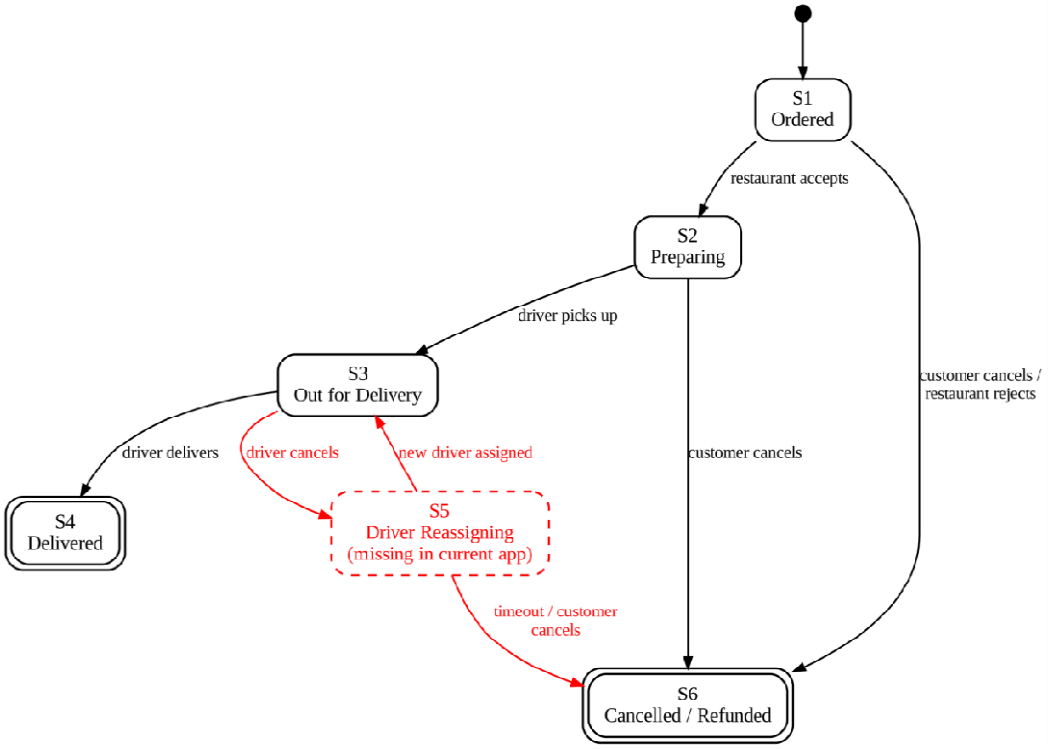


Figure 10.1: State transition diagram with the missing driver reassignment state (red)

I added two states to the four given ones: **S5 Driver Reassigning** (driver cancelled, waiting for a new driver) and **S6 Cancelled / Refunded**.

State transition table (x = invalid event, the system must ignore it or show an error):

State	E1 Restaurant accepts	E2 Customer cancels	E3 Driver picks up	E4 Driver delivers	E5 Driver cancels	E6 New driver assigned	E7 Timeout
S1 Ordered	S2	S6	x	x	x	x	x
S2 Preparing	x	S6	S3	x	x	x	x
S3 Out for Delivery	x	S3*	x	S4	S5	x	x
S4 Delivered	x	x	x	x	x	x	x
S5 Driver Reassigning	x	S6	x	x	x	S3	S6
S6 Cancelled	x	x	x	x	x	x	x

* The customer cannot cancel once the food is on the way. The request is rejected and the order stays in S3.

4. Redesigned test scenarios

The table has 6 x 7 = 42 cells. 10 are defined behaviour (9 state changes and 1 rejected cancel). The other 32 are invalid and must be rejected.

ID	Type	Sequence of events	Expected result
TC1	Valid	S1 -E1-> S2 -E3-> S3 -E4-> S4	Delivered (normal flow)
TC2	Valid	S1 -E2-> S6	Cancelled, full refund
TC3	Valid	S1 -E1-> S2 -E2-> S6	Cancelled, refund (fee if any)
TC4	Valid	S3 -E5-> S5 -E6-> S3 -E4-> S4	Reassigned and delivered (the reported defect)
TC5	Valid	S3 -E5-> S5 -E7-> S6	Cancelled, refund, user notified
TC6	Valid	S3 -E5-> S5 -E2-> S6	Cancelled, refund
TC7	Valid	In S3, E2	Cancel rejected, state stays S3
TC8	Sequence	S3 -E5-> S5 -E6-> S3 -E5-> S5 -E6-> S3 -E4-> S4	Driver cancels twice, still delivered
TC9	Invalid	In S1, E4 (deliver)	Rejected, stays S1
TC10	Invalid	In S1, E3 (pickup)	Rejected, stays S1
TC11	Invalid	In S4, E2 (cancel after delivery)	Rejected, stays S4
TC12	Invalid	In S6, E1 (accept)	Rejected, stays S6
TC13	Invalid	In S2, E5 (driver cancels before pickup)	Rejected, stays S2
TC14	Invalid	In S3, E6 (new driver while one is assigned)	Rejected, stays S3
TC15	Recovery	App closed and reopened while in S5	Tracker shows “Finding a new driver”

TC1 to TC7 cover all 10 defined transitions (100% transition coverage). TC8 is a longer sequence (1-switch coverage). Since the table is small, all 32 invalid cells can be run automatically with a loop over states and events, and only TC9 to TC14 are shown above as examples.

5. Why State Transition Testing finds what ECP and BVA miss

Point	ECP / BVA	State Transition Testing
What it looks at	The value of one input	The current state and the event together

Point	ECP / BVA	State Transition Testing
History	Ignored. Each test is separate	The result depends on the sequence that came before
Example: "cancel"	It is just a valid input, one test is enough	It gives different results in S1, S2, S3, S4, S5 and S6
Missing states	Cannot show that a state is missing	Drawing the model makes the gap visible (no state for driver cancel)
Invalid actions	Only invalid values	Invalid event for a state (cancel after delivery)
Boundaries	Numeric limits	There are no numbers to test here, so BVA has nothing to work on

In a dynamic workflow the same input can be right in one state and wrong in another. State transition testing makes this explicit, and that is how the stuck tracker is found and fixed.

Answer 11

1. Understanding the scenario

The login needs three things: a strong password, a correct OTP entered in time, and an acceptable IP location. Brute-force weaknesses were still found after testing. A brute-force attack works by trying very many passwords or OTPs. Protection against it (attempt counters, account lock, OTP invalidation) is code that runs only when something goes wrong, so normal functional tests never reach it.

2. Critical assessment of the black-box and white-box methods used

Method used	What it did well	What it missed
Black-box (ECP/BVA)	Checked correct and wrong password, OTP at its time limit, normal IP	Tested each factor once. Never repeated the failure many times, so counters were never checked
White-box (statement coverage)	Executed the main login code	Lines such as lock(user) and invalidate(otp) were never reached, since they need 5 or 3 failures in a row
Integration of the two	Both were used	The black-box tests came from the requirement and the white-box tests from the code, but nobody looked at the attacker view

3. Login logic and Control Flow Graph

```
[1]  read(user, pwd, otp, ip)
[2]  if (locked(user))
[3]      return LOCKED
[4]  if (pwdCorrect(user, pwd)) {
[9]    if (ipBlocked(ip))
[10]       return DENY
[11]   if (otpCorrect(user, otp))
[16]       return GRANT   // OTP: 60 s
[12]   otpFails++
[13]   if (otpFails >= 3)
[14]       invalidate(otp)
[15]   return DENY
     } else {
[5]    fails++
[6]    if (fails >= 5)
[7]        lock(user)
[8]    return DENY
     }
```

Decisions: nodes 2, 4, 6, 9, 11, 13.
T/F on an arrow is the answer to the question written for that node.

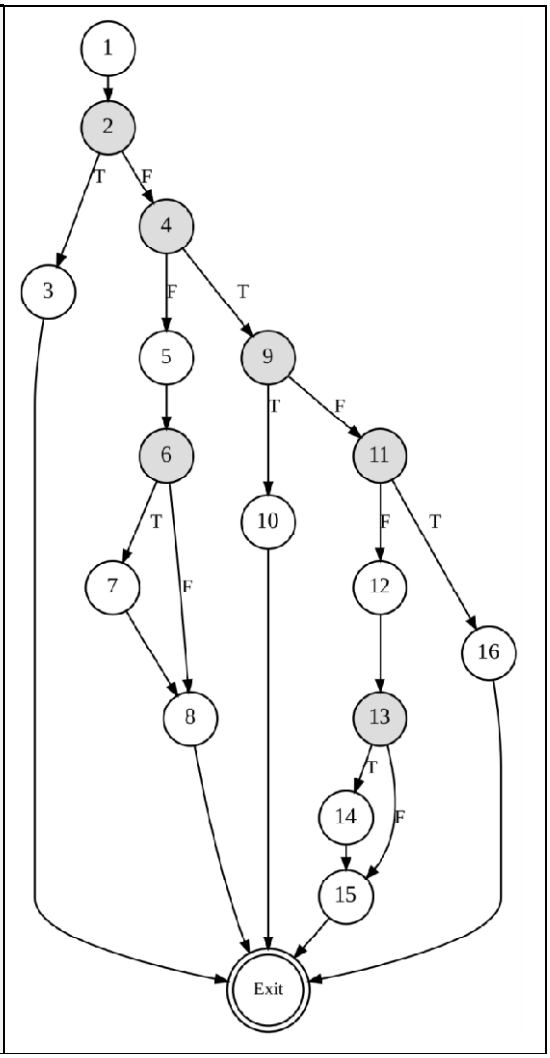


Figure 11.1: CFG of the authentication module

Six decisions, so **V(G) = 6 + 1 = 7** independent paths.

Path	Scenario	Covered earlier?	Expected result
P1	Account already locked	No	LOCKED, no further checks
P2	Wrong password, fewer than 5 failures	Yes	DENY, counter increases
P3	Fifth wrong password in a row	No	Account is locked
P4	Correct password, blocked IP	No	DENY
P5	Wrong OTP, fewer than 3 failures	Yes	DENY, OTP counter increases

Path	Scenario	Covered earlier?	Expected result
P6	Third wrong OTP	No	OTP invalidated, a new one is required
P7	All three factors correct	Yes	GRANT

Only 3 of 7 paths were tested. The 4 paths that were missed include both brute-force protections: the account lock (P3) and the OTP limit (P6). So the defence code was never run during testing.

4. Proposed Hybrid Testing strategy

The hybrid approach uses error guessing to think like an attacker and formal path analysis to make sure that each guess is aimed at a real path in the code.

Step	Technique	What is tested
1	Black-box functional (ECP, BVA)	Password length 7 / 8, weak / medium / strong password, OTP at 59 s, 60 s and 61 s, trusted / new / blocked IP
2	White-box path analysis	All 7 basis paths (P1 to P7), plus MC/DC for each condition that combines two checks
3	Error guessing	Attack-style guesses listed below, each mapped to a path
4	Coverage check	Branch and path coverage measured; any path not reached gets a new test

Error guesses and the path each one checks (to be run only in the team's own test environment):

Guess	Path	Secure behaviour expected
5 wrong passwords in a row, then the correct one	P3, P1	Account locked, even the correct password is refused
Many OTP guesses inside the 60 s window	P6	OTP is cancelled after 3 failed tries
Reusing an OTP that was already accepted	P5	Rejected
OTP entered at 61 s	P5	Rejected (expired)
Counter reset by logging in from different IPs, or parallel requests	P3, P6	Counters are per user and are not bypassed

Guess	Path	Secure behaviour expected
Login from a blocked IP with a correct password	P4	DENY
Error message that tells which factor failed	P2, P5	Same generic message for every failure
Very long or special-character username and password	P2	Handled safely, no crash

5. How this improves the security posture

- **Defence code is actually executed.** All 7 paths run, including lockout and OTP invalidation, so a broken counter is found in testing.
- **Attacks are made unlikely by numbers.** A 6-digit OTP has 1,000,000 possible values. With 3 attempts allowed, the chance of guessing is 3 / 1,000,000 = 0.0003%. Without a limit, thousands of guesses per minute can succeed.
- **Information leaks are checked.** Uniform error messages stop an attacker from learning which factor was correct.
- **Coverage gives evidence.** Showing that 100% of the paths and the listed attack guesses were tested supports security reviews and audits.
- **Repeatable.** The cases are automated and re-run after every change to the login code.

Answer 12

1. Understanding the scenario

Access to courses depends on the plan (Free, Basic, Premium) and on the subscription dates. The error occurs when a user upgrades exactly on the expiry date. This is a date boundary problem combined with a plan change. The question does not give the rules, so I assumed these:

- A subscription is valid up to and including its expiry date E (until 23:59:59). From E + 1 the user is treated as Free.
- An upgrade on or before E starts the new plan immediately, with no gap in access.
- An upgrade after E is a fresh subscription, and earlier course progress is kept.

Plan	Free courses	Basic courses	Premium courses
Free (or expired)	Allowed	Denied	Denied
Basic (active)	Allowed	Allowed	Denied
Premium (active)	Allowed	Allowed	Allowed

2. Evaluation of the current testing criteria

- Tests used dates well inside the subscription period (for example 10 days before expiry), never the expiry day itself.
- Each plan was checked on its own, but the change of plan (upgrade or renewal) was not checked at different dates.
- There was no test for the last second of the expiry day or the first second after it, so the time part of the boundary was not tested.
- There was no criterion such as "every date class × every plan change has a test".

3. Equivalence classes

Dimension	**Class**	**Representative value**
Plan	P1 Free (or never subscribed)	New user
Plan	P2 Basic	Basic subscriber
Plan	P3 Premium	Premium subscriber
Date vs. expiry E	D1 Active, before expiry (today < E)	E - 10 days, E - 1 day
Date vs. expiry E	D2 Expiry day (today = E)	E at 00:00:00, 10:00, 23:59:59
Date vs. expiry E	D3 Expired (today > E)	E + 1 day at 00:00:00, E + 30 days
Course type	C1 Free, C2 Basic, C3 Premium	One course of each type
Action	A1 Open course, A2 Upgrade, A3 Renew same plan, A4 Downgrade	One of each

The boundary values for dates are E - 1, E, E + 1, and for the time of day 00:00:00 and 23:59:59 on E. The user's time zone and the server time zone should also be the same on the expiry day, or the user may lose access hours early.

4. Structured test design

ID	**Current plan**	**Date**	**Action**	**Expected result**
U1	Free	-	Upgrade to Basic	Basic active from today
U2	Free	-	Upgrade to Premium	Premium active from today
U3	Basic	E - 10 (D1)	Upgrade to Premium	Premium courses open immediately
U4	Basic	E - 1 (D1)	Upgrade to Premium	Premium courses open immediately

ID	Current plan	Date	Action	Expected result
U5	Basic	E, 10:00 (D2)	Upgrade to Premium	Premium opens, no drop to Free, no double charge
U6	Basic	E, 23:59:59 (D2)	Upgrade to Premium	Premium opens, access has no gap
U7	Basic	E + 1, 00:00:00 (D3)	Upgrade to Premium	Treated as new subscription, Premium opens
U8	Premium	E, 10:00 (D2)	Renew Premium	No break in access, new expiry date set
U9	Premium	E, 10:00 (D2)	Downgrade to Basic	Basic access, Premium courses locked
U10	Basic	E + 30 (D3)	Renew Basic	Treated as new subscription
U11	Basic	E (D2)	Open Basic course	Allowed (still active on expiry day)
U12	Basic	E + 1 (D3)	Open Basic course	Denied, Free access only
U13	Basic	E (D2)	Open Premium course	Denied, upgrade offer shown
U14	Premium	E + 1 (D3)	Open Premium course	Denied, renewal offer shown

5. Why the design is complete

Dimension	Classes	Covered by
Plans	Free, Basic, Premium	U1, U2 (Free); U3 to U7, U10 to U13 (Basic); U8, U9, U14 (Premium)
Date classes	D1, D2 (3 time points), D3	U3, U4 (D1); U5, U6, U8, U9, U11, U13 (D2); U7, U10, U12, U14 (D3)
Plan changes	Upgrade, renew, downgrade	Upgrade: U1 to U7; renew: U8, U10; downgrade: U9
Course access	Allowed and denied for each plan	U11 to U14, and the access table above
Boundaries	E - 1, E, E + 1, 00:00:00, 23:59:59	U4, U5, U6, U7

Every plan, every date class and every kind of plan change appears in at least one test, and the expiry day is tested at the start, middle and end. So the seamless transition (no loss of access and no double charge) is checked in all the ways a user can reach it.

Answer 13

1. Understanding the scenario

The lighting controller reads the ambient light (lux) and decides how bright the lamps should be. The code has a switch on the mode and compound Boolean expressions. A conceptual version is shown below.

```
int dim(int mode, int lux, boolean occupied, boolean night) {
   int brightness;                    // not initialised
   switch (mode) {
      case AUTO:
         if (occupied && (lux < 200 || night))
            brightness = 100 - lux / 4;
         else
            brightness = 0;
         break;
      case MANUAL:
         brightness = userLevel;
         break;
      case ECO:
         brightness = (lux < 100) ? 40 : 0;
         break;
   }                                  // no default case
   return brightness;                 // sent to the lamp driver
}
```

2. Risks of relying only on statement coverage

Four tests are enough to execute every statement: (AUTO, occupied, lux 150, night false), (AUTO, not occupied), (MANUAL) and (ECO, lux 50). That is 100% statement coverage, yet:

- The case where mode matches nothing is never tried, so brightness is returned uninitialised and the lamp driver receives a random value.
- The ternary (lux < 100) ? 40 : 0 is one statement but has two outcomes. Only the first was run.
- In occupied && (lux < 200 || night), the cases lux >= 200 with night = true, or night = false, were never tried.
- The formula 100 - lux / 4 becomes negative for lux above 400, which statement coverage cannot reveal.

Measure	Result with those 4 tests	Missed
Statement coverage	100%	-
Branch coverage	6 / 8 = 75%	Switch with no matching case, ternary false
Condition coverage	4 / 6 = 66.7%	lux < 200 false, night true

Branches counted: 3 switch cases + 1 no-match case, if true / false, ternary true / false = 8. Condition outcomes counted: occupied (T, F), lux < 200 (T, F), night (T, F) = 6.

3. Applying Branch and Condition Coverage

ID	Mode	lux	occupied	night	Expected brightness	Purpose
T1	AUTO	150	T	F	63	if true via lux < 200
T2	AUTO	500	T	T	0 (limit to 0)	lux < 200 false, night true
T3	AUTO	500	T	F	0	if false, both parts false
T4	AUTO	100	F	T	0	occupied false
T5	MANUAL	-	-	-	userLevel (70)	MANUAL case
T6	ECO	50	-	-	40	Ternary true
T7	ECO	100	-	-	0	Ternary false, boundary of < 100
T8	invalid (99)	-	-	-	Safe default (0)	No matching case
T9	MANUAL	-	-	-	100 for userLevel 120	Output must stay in 0 to 100

These tests reach 8 / 8 branches and all 6 condition outcomes. They also expose two defects that the earlier tests could not: T2 returns 100 - 125 = **-25** (a negative value sent to the lamp) and T8 returns garbage because of the missing default.

4. Improved white-box strategy

1. Draw the logic of the function and list every decision (switch, if, ternary) and every condition.
2. Set the target: 100% statement, 100% branch and 100% condition coverage. For the safety-related output, MC/DC can be added on the occupied && (...) decision.

3. Write test cases from the table above, including a default case for the switch and range checks (0 to 100) on the output.
4. Measure coverage with a tool and add tests until the target is met.
5. Fix the defects found: initialise brightness, add a default case and limit the result to the range 0 to 100.

5. How this reduces hardware-software integration failures

- The driver only receives values from 0 to 100, because range and default cases are tested. An out-of-range value could otherwise cause flicker or damage to the lamp.
- Each combination of sensor reading and mode, including the boundary lux values, has a known expected result, so problems at the sensor-to-software interface are found in software tests before real hardware is used.
- Uninitialised variables and missing cases, which behave differently on different boards, are removed.
- The tests run without hardware, so they can be repeated cheaply after every change.

Answer 14

1. Understanding the scenario

The shipping cost depends on weight, destination zone and delivery speed. The testers did not find errors when the weight was zero or negative. I assumed the following rules:

Item	**Assumption**
Valid weight	0.1 kg to 50 kg, up to 2 decimal places
Zones and base charge	Z1 = Rs. 50, Z2 = Rs. 80, Z3 = Rs. 120, Z4 = Rs. 300
Speed and rate per kg	Standard = Rs. 10, Express = Rs. 20, Overnight = Rs. 40
Formula	Cost = base charge (zone) + weight x rate (speed)

2. Analysis of the failure

- The input validation tests used only normal positive weights such as 1, 5 or 10 kg. Zero and negative values were never entered.
- Without a check, the formula accepts them. A negative weight reduces the price. For example Z2 with Standard speed and weight -10 gives 80 + (-10 x 10) = **-20**, a negative shipping cost.
- A weight of 0 gives a price for a parcel that does not exist (Rs. 80 here), which can also hide bad data coming from another system.
- Validation was probably checked only in the user interface, so other clients (mobile app, API) can send bad values.

3. Boundary Value Analysis with robustness testing

Normal BVA tests only inside the valid range (min, min + 1, nominal, max - 1, max). **Robustness testing** adds the values just outside the range (min - 1, max + 1), so that the invalid boundaries are also tested.



Figure 14.1: Robustness test points around the minimum and maximum weight

Test values use Z2 and Standard speed (base 80, rate 10 per kg). The last column shows what the system would give today if no validation exists.

ID	Weight (kg)	Class	Expected result	Result with no validation
W1	-10	Invalid (negative)	Rejected with message	-20 (negative cost)
W2	-1	Invalid (negative)	Rejected with message	70
W3	0	Invalid (min - 0.1)	Rejected with message	80 (charged for no weight)
W4	0.1	Min	Rs. 81.00	81
W5	0.2	Min + 1 step	Rs. 82.00	82
W6	25	Nominal	Rs. 330.00	330
W7	49.9	Max - 1 step	Rs. 579.00	579
W8	50	Max	Rs. 580.00	580
W9	50.1	Invalid (max + 1 step)	Rejected with message	581 (over the limit accepted)
W10	abc / blank	Invalid (not a number)	Rejected with message	Error or crash
W11	999999999	Invalid (extreme)	Rejected with message	Huge price or overflow
W12	0.123	Invalid (3 decimals)	Rejected or rounded as per rule	81.23

4. Other inputs and combinations

- **Zone:** one test for each valid zone (Z1 to Z4) and an invalid code (Z9, blank).
- **Speed:** one test for each valid speed (Standard, Express, Overnight) and an invalid value ("Instant").
- **Valid combinations:** 4 zones x 3 speeds = 12 tests at the nominal weight of 25 kg, for example Z1 + Express = 50 + 25 x 20 = 550.
- **Invalid weight with every speed and zone type:** at least one test such as Z4 + Overnight with weight -5, to show that rejection does not depend on the other inputs.

5. Why negative scenarios must be tested

Negative scenario	**What can happen in production**
Negative or zero weight	Free or negative shipping, direct loss of money, or fraud by deliberate entry
Weight above the maximum	Orders that the carrier cannot deliver, wrong cost
Non-numeric or empty input	Crash or unclear error for the user
Extremely large value	Overflow, slow processing, possible denial of service
Wrong zone or speed code	Wrong rate used silently

- Real users and other systems do make mistakes, so the software must fail safely and give a clear message.
- Fixing a defect found in testing costs far less than fixing it after money has been lost.
- Validation must be checked on the server side as well, since the user interface can be bypassed.
- A system that handles invalid input correctly is more stable and trusted, which is the meaning of robustness.

Answer 15

1. Understanding the scenario

The imaging software splits n data elements between T parallel workers. Results are wrong only when n is a prime number. A prime number has no divisor other than 1 and itself, so for any worker count T with $1 < T < n$, the division n / T always leaves a remainder. Therefore prime sizes always use the code that handles the leftover elements. Sizes the team normally used (powers of two such as 256 or 512) divide evenly by T, so the leftover code was never run. Below is a conceptual version with the kind of defect that fits this behaviour.

2. Code and Control Flow Graph

```
[1] chunk = n / T; rem = n % T
[2] for (t = 0; t < T; t++)
[3]   for (i = t*chunk;
          i < (t+1)*chunk; i++)
[4]       result[i] = filter(data[i])
[5]   (end of one worker, t++)
[6] if (rem != 0)
[7]   for (i = n - rem; i < n - 1; i++)
          // BUG: should be i < n
[8]       result[i] = filter(data[i])
[9] output(result)
```

Decisions: nodes 2, 3, 6, 7.
Red dashed edge 7 -> 9: the leftover loop stops one element too early, so the last element is never processed.

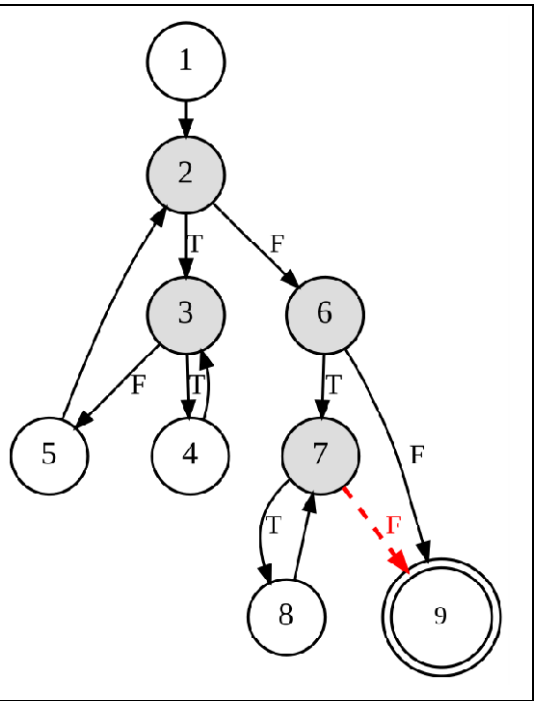


Figure 15.1: CFG of the parallel processing module

E = 12, N = 9, so **V(G) = 12 - 9 + 2 = 5**. Check: 4 decisions + 1 = 5. Whenever n is not a multiple of T, exactly one element is left unprocessed. That element keeps whatever value was in memory, so each run can give a different answer, which is why the results look inconsistent.

3. Evaluation of random testing

Advantages of random testing

- Cheap to produce and good at finding unexpected crashes.
- Can explore many data values without much thinking.

Disadvantages for this module

- It does not know the mathematics of the code. The special classes are n divisible by T, n with remainder 1, n less than T, and prime n. Random choices from typical image sizes never reach most of them.
- There is no measure of how much of the code was covered, so the team could not tell that the leftover loop had never run.
- Parallel code is hard to reproduce. A random failure that depends on thread timing may not happen again, and the seed or thread count may not have been recorded.
- There is no clear expected result for a random input unless a trusted reference exists.

So for a specialized mathematical module, random testing should only be a supporting tool. The main design should be based on the structure of the code.

4. Structured White-Box strategy with Basis Path Testing

Five basis paths are needed. Each test below is chosen to add one new part of the graph. The tests use T = 4 workers.

Path	Route	n	Why this n	Expected	With the bug
P1	1-2-6-9	any	Invalid worker count T = 0	Rejected before dividing	Division by zero
P2	loops run, 6 false	8	n divisible by T (rem 0)	8 of 8 processed	8 of 8 (passes)
P3	inner loop skipped, leftover loop used	3	Prime, n less than T (chunk = 0)	3 of 3 processed	2 of 3
P4	both loops, leftover loop runs	11	Prime (rem 3)	11 of 11 processed	10 of 11
P5	leftover loop does not run	9	rem = 1, loop body empty	9 of 9 processed	8 of 9

Extra boundary tests on n and T: n = 0 (empty), n = 1, n = 10 (rem 2), n = 97 (large prime), and several values of T (1, 2, 3, 4, 8, 16). A fixed list of such cases covers the classes that random selection misses.

- **Test oracle:** the output of the parallel code is compared with a simple sequential version for the same input. They must be identical.
- **Invariant check:** every index from 0 to n - 1 must be processed exactly once. Counting the processed indexes shows the defect immediately. Each case is also run several times to catch inconsistent results.

The corrected line is for (i = n - rem; i < n; i++). After the fix, all of the above tests pass, and they stay in the regression suite.

5. Justification

- **Mathematical accuracy:** the tests are built around the arithmetic that splits the data (division, remainder, empty chunk), so each part of the calculation is checked, and the result is compared with a trusted reference.
- **Coverage:** 5 basis paths (V(G) = 5) plus boundary values on n and T give a measurable level of coverage, which random testing could not give.
- **Stability and reproducibility:** the same input gives the same output on every run and for every worker count, and each test has a fixed input, so any failure can be repeated and debugged.
- **Risk:** in medical imaging, one wrong pixel or slice can lead to a wrong diagnosis. Testing the special sizes (primes, remainders, small sizes) removes the failure before release.